Práctica p6-Extra Iker Núñez UO306425

Se pedía codificar un algoritmo que resolviera el problema de las nReinas. El código resultante fue el siguiente:

Por una parte, el código del algoritmo:

```java
public class NReinas {

    private List<List<String>> soluciones; // cada lista interna de Strings representa un tablero completo donde cada Strir

    private char[][] tablero;
    private boolean[] columnas; // indica si una columna específica (de 0 a N-1) ya tiene una reina.
    private boolean[] diagonalesPrincipales; // de arriba a abajo, izda a decha (\)
    private boolean[] diagonalesSecundarias; // de arriba a abajo, decha a izda (/)

    /**
     * Método principal que inicia el algoritmo.
     * @param n de tipo int. El tamaño del tablero y el número de reinas.
     * @return soluciones, de tipo List<List<String>>. Una lista con todas las configuraciones válidas del tablero
     */
    public List<List<String>> resolverNReinas(int n) {
        soluciones = new ArrayList<>();

        // Inicializamos los rastreadores de ataques en columnas y diagonales
        columnas = new boolean[n];
        diagonalesPrincipales = new boolean[2*n-1];
        diagonalesSecundarias = new boolean[2*n-1];

        // Inicializar el tablero vacío con puntos '.' (os doy un método)
        tablero = crearTablero(n);
        // Iniciar el backtracking
        backtracking(0, n, tablero);

        return soluciones;
    }
```

```java
 * Método de backtracking para colocar las reinas en el tablero.
 * @param fila, de tipo int. La fila actual en la que intentamos colocar una reina.
 * @param n,    de tipo int. El tamaño del tablero y el número de reinas.
 * @param tablero,  de tipo char[][]. La representación actual del tablero con las reinas colocadas.
 */
private void backtracking(int fila, int n, char[][] tablero) {
    // Caso base: si logramos llegar a la fila n, colocamos todas las reinas con éxito
    if(fila == n){
        List<String> solucion = construirSolucion(tablero);
        soluciones.add(solucion);
        return;
    }

    for (int col = 0; col < n; col++) {

        // Fórmulas matemáticas para identificar la diagonal a la que pertenece una celda
        int d1 = fila - col + (n - 1); // Diagonal principal (\)
        int d2 = fila + col;           // Diagonal secundaria (/)

        // 1. Avanzar -> Colocar la reina (meter un 'Q') y marcar la columna y diagonales como atacadas
        if(columnas[col] || diagonalesPrincipales[d1] || diagonalesSecundarias[d2]){
            continue;
        }

        columnas[col] = true;
        diagonalesPrincipales[d1] = true;
        diagonalesSecundarias[d2] = true;
        tablero[fila][col] = 'Q';
```

```java
for (int col = 0; col < n; col++) {

    // Fórmulas matemáticas para identificar la diagonal a la que pertenece una celda
    int d1 = fila - col + (n - 1); // Diagonal principal (\)
    int d2 = fila + col;           // Diagonal secundaria (/)

    // 1. Avanzar -> Colocar la reina (meter un 'Q') y marcar la columna y diagonales como atacadas
    if(columnas[col] || diagonalesPrincipales[d1] || diagonalesSecundarias[d2]){
        continue;
    }

    columnas[col] = true;
    diagonalesPrincipales[d1] = true;
    diagonalesSecundarias[d2] = true;
    tablero[fila][col] = 'Q';

    // 2. backtracking
    backtracking(fila + 1, n, tablero);

    // 3. Retroceder
    columnas[col] = false;
    diagonalesPrincipales[d1] = false;
    diagonalesSecundarias[d2] = false;
    tablero[fila][col] = '.';
}
```

Y por otra parte, una clase que mide los tiempos de ejecución:

```java
public class NReinasTiempos{
    private static NReinas nReinas = new NReinas();

    Run | Debug
    public static void main(String[] args){
        long t1;
        long t2;
        double tFinal;

        System.out.println(x: "MEDICION DE TIEMPOS N REINAS: ");

        for(int i = 2; i < 20; i += 2){
            t1 = System.nanoTime();

            nReinas.resolverNReinas(i);

            t2 = System.nanoTime();

            tFinal = (t2 - t1)/ 1_000_000;

            System.out.printf(format: "PARA N = %d , T = %f MS", i, tFinal);
            System.out.println();
        }

        System.out.println(x: "FIN DE LA MEDICION DE TIEMPOS");
    }
}
```

El resultado de la medición de tiempos fue la siguiente:

n	t(ms)
2	0
4	0
6	0
8	2
10	5
12	127
14	4013
16	OutOfMemError